

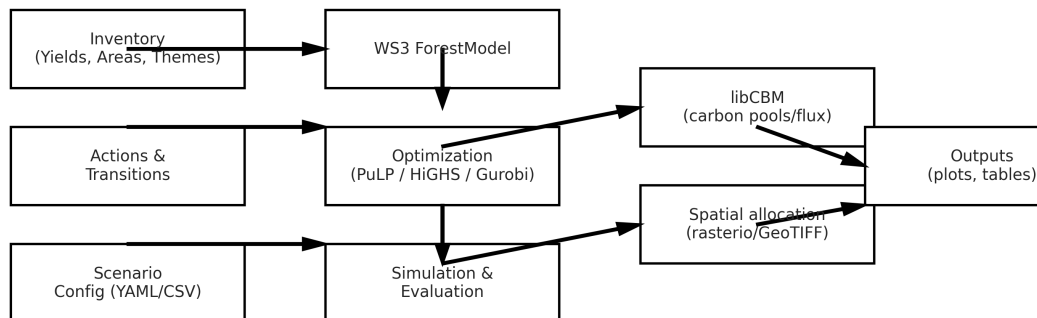
Highlights

- An open-source Python framework for decision support in forest landscape and wood supply planning.
- Modular architecture with solver-backed optimization and hybrid spatial/aspatial workflows.
- Interoperable with standard geospatial formats and supports carbon accounting via libcbm.
- Reproducible workflows, CI-tested codebase, and PyPI distribution.
- Demonstrated on a real-world case study with transparent, reusable notebooks and scripts.

Graphical Abstract

WS3: An open-source Python framework for integrated simulation and optimization of forest landscape and wood supply systems

Gregory Paradis



WS3: An open-source Python framework for integrated simulation and optimization of forest landscape and wood supply systems

Gregory Paradis^{a,*}

^a*Faculty of Forestry, University of British Columbia, Canada*

Abstract

Transparent, reproducible decision support for forest landscape planning increasingly requires integrated treatment of harvest scheduling, spatial allocation, and greenhouse gas (GHG) accounting. We present WS3, an open-source Python framework for long-horizon wood supply modeling that combines modular simulation with solver-backed optimization and hybrid spatial/aspatial workflows. WS3 represents stands, actions, and transitions through a clear data model, supports scenario-based analyses, and interoperates with geospatial rasters for spatial allocation of aspatial schedules. A key contribution is a built-in linkage to the open-source Canadian Forest Service Carbon Budget Model (libCBM), enabling seamless carbon stock and flux estimation alongside traditional harvest and area outcomes. We demonstrate WS3 on a reproducible case study that couples optimization-driven harvest scheduling with libCBM carbon accounting and a raster allocation step to visualize spatial implications. Results highlight how alternative policy constraints influence harvest flows and carbon dynamics, illustrating the framework's utility for climate change mitigation planning. The full code, documentation, and notebooks are publicly available via GitHub and PyPI, with a versioned archive on Zenodo, and a complete reproduction package accompanying this article. WS3 lowers barriers to rigorous, extensible analysis, supporting researchers and practitioners seeking open, auditable workflows for forest sector planning under evolving climate and policy objectives.

Keywords: Environmental modelling, Decision support, Forest estate, Optimization, Simulation, Spatial modelling, Reproducibility, Open-source software

*Corresponding author: gregory.paradis@ubc.ca

1. Introduction

Forest landscape planning and wood supply analysis underpin long-term decisions about harvest allocations, silviculture, and conservation. Yet practitioners and researchers frequently face fragmented toolchains and proprietary software that limit transparency, extensibility, and reproducibility—issues at the heart of open predictive ecology and the PERFICT principles [10]. These constraints are particularly acute when planning under climate objectives, where greenhouse gas (GHG) accounting must be integrated into scenario design, evaluation, and communication [17].

We introduce WS3, an open-source Python framework for integrated simulation and optimization of forest landscape and wood supply systems. WS3 supports long-horizon, multi-scenario analyses; integrates solver-backed harvest scheduling; and provides a hybrid spatial/aspatial workflow that maps aspatial schedules to raster inventories for spatial evaluation. A defining feature is its built-in linkage to the Canadian Forest Service `libcbm` library, enabling carbon stock and flux estimation directly within planning workflows [9, 2].

This article makes three contributions: (i) a description of WS3’s modular architecture and data model designed for clarity and reuse; (ii) a reproducible case study that couples optimization with libCBM-based carbon accounting and spatial allocation; and (iii) a comparison with established tools highlighting openness, extensibility, and cross-ecosystem interoperability. We provide a versioned release with an accompanying reproduction package to facilitate verification and reuse.

2. Software description

2.1. Architecture

WS3 implements a modular architecture organized around five roles: (i) core data abstractions; (ii) forest model construction and simulation; (iii) optimization; (iv) spatial allocation; and (v) common utilities. The main modules are:

- `forest`: defines the `ForestModel` and development types, actions, transitions, yields, and scenario compilation. A scenario is specified by a planning horizon, period length, initial inventory, and transition rules that determine stand state changes under actions and growth.
- `opt`: provides a solver-agnostic interface to formulate linear programming harvest-scheduling problems (objective, variables, constraints) and to solve them using PuLP/HiGHS by default, with optional Gurobi.

- *spatial*: implements utilities to map aspatial schedules to rasters (e.g., GeoTIFF) for visualization and spatial policy evaluation.
- *core/common/financial/helper*: shared utilities for interpolation, curve handling, financial indicators, and helper logic used by examples and workflows.

Data model and workflow. Inputs include initial inventory (areas by development type and age), yield curves, action definitions (e.g., harvest, silviculture, fire), and transition rules. WS3 builds a state-transition tree per development type across periods and supports both heuristic scheduling (priority-queue area selection) and optimization-based scheduling. Outputs include time-series indicators (harvested area/volume, growing stock), compiled action schedules, and optional spatial rasters.

Interoperability. WS3 is designed to be explicit and auditable: configuration and tabular inputs are plain text (CSV/TSV), and geospatial outputs use standard rasters (GeoTIFF). A built-in linkage exports a Standard Input Table (SIT) data format compatible with libCBM [9, 2], enabling carbon stock and flux estimation without ad hoc adapters.

Figure 1 summarizes modules and data flow; Figure 2 shows the end-to-end workflow used in the case study.

2.2. Implementation

Technology stack. WS3 is implemented in Python and distributed via PyPI. Optimization problems are formulated with PuLP [11] and solved with open-source HiGHS [6] by default, with optional Gurobi [5] for larger instances. Geospatial I/O uses rasterio/fiona/GeoPandas. Documentation (Sphinx) and continuous integration ensure API consistency, example executability, and unit testing.

libCBM integration. The ForestModel exposes a `to_cbm_sit` method that compiles a SIT configuration and tables (classifiers, inventory, yields, disturbance events, transitions) consumable by libCBM. Users provide a disturbance type mapping and last-pass disturbance metadata to ensure correct dead organic matter (DOM) spin-up. The case study runs libCBM for 200 years using the official Python API.

Reproducibility and packaging. The repository includes examples and a reproduction package (papers/ems/repro) with pinned requirements and scripts to generate all figures and tables. Ver-

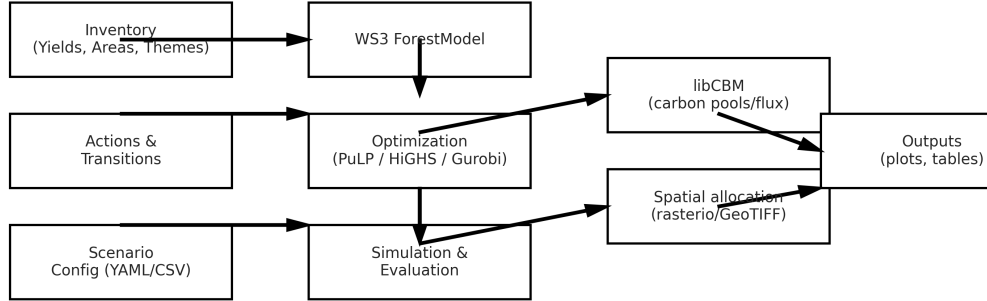


Figure 1: WS3 architecture: inputs feed a ForestModel; scenarios are scheduled via heuristic or LP; results feed libCBM and optional spatial allocation; plots/tables generated for analysis.

sioned releases are archived on Zenodo; the EMS-paper release DOI is referenced in this manuscript. WS3 supports interactive (Jupyter) and batch (Linux) workflows.

Performance and scalability. Typical case studies with tens of thousands (to millions) of stands and multi-decade-length planning horizons solve in seconds to minutes with HiGHS on a modern workstation. Larger landscapes and constraint sets can benefit from Gurobi (e.g., to leverage built-in feasibility-restoration functions via automated LP problem reformulation as a weighted goal-programming problem—not available yet when using the default open-source solver backends) and batching. Spatial allocation steps scale with raster size but are embarrassingly parallel across periods.

3. Illustrative case study

We demonstrate WS3 using a two-stage sequential pipeline that first schedules harvest in WS3 and then evaluates carbon dynamics using libCBM. The workflow replicates the existing example 031_ws3_libcbm_sequential-builtin.ipynb to ensure full reproducibility.



Figure 2: Case study workflow: load inputs, build ForestModel, schedule, export SIT (to_cbm_sit), run libCBM, generate plots and tables.

Study design. We load an inventory (areas, yields, actions, transitions) and schedule harvest using a self-parameterizing area-control heuristic, producing an aspatial action schedule over a 100-year horizon (10 periods of 10 years). We then export a libCBM SIT with a disturbance mapping for harvest and fire and run libCBM for 200 annual steps to estimate stocks and fluxes in biomass and dead organic matter pools.

Outputs. We report time-series of harvested area/volume and growing stock from WS3, and annual carbon stocks from libCBM. The reproduction package generates the figures and tables used below.

The full, deterministic workflow is scripted in `papers/ems/repro/generate_case_study.py`, which produces the flows table and carbon stocks table:

- `papers/ems/tables/scenario_flows.csv`: harvest area, harvest volume, and growing stock by period.
- `papers/ems/tables/annual_carbon_stocks.csv`: annual biomass, DOM, and total ecosystem carbon.

Figures 4 and 5 visualize the resulting flows and carbon stocks. The entire pipeline, including architecture and workflow diagrams (Figures 1–2) and a schematic spatial placeholder (Figure 3), is reproducible via `papers/ems/repro/make_repro.sh`. The libCBM run length (200 years) matches the example and can be adjusted.

al allocation (schematic stub): harvested cells in one period (·

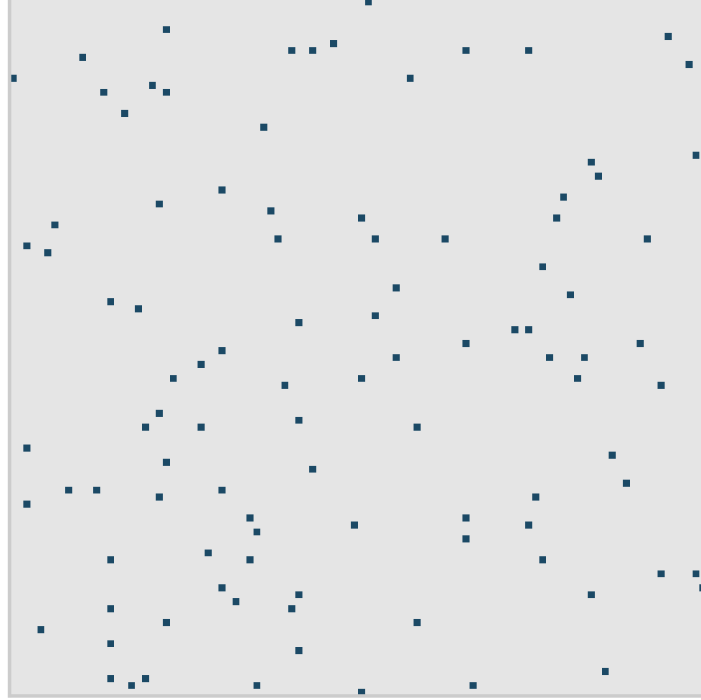


Figure 3: Spatial allocation schematic (stub). A stylized grid highlights an illustrative $\sim 1\%$ of cells as ‘harvested’ in one period, emphasizing the small spatial footprint typical of long-horizon planning. This placeholder will be replaced with a map derived from the raster allocation in the final submission.

Reproducibility. This case is intended to be transparent and portable; data are included in the repository’s examples folder. The spatial allocation step is kept illustrative; WS3’s hybrid spatial/aspatial workflow supports allocation of aspatial schedules to raster inventories, and we will provide a finalized spatial figure in the submission package. The folder papers/ems/repro contains a `make_repro.sh` script that creates a virtual environment, installs pinned dependencies, and runs a deterministic Python script to regenerate Figures 4–5 and associated CSV tables.

4. Impact and reusability

WS3 enables transparent, auditable analysis for forest landscape planning and climate mitigation. It has supported research on bioenergy [3], value-creation potential and supply modelling

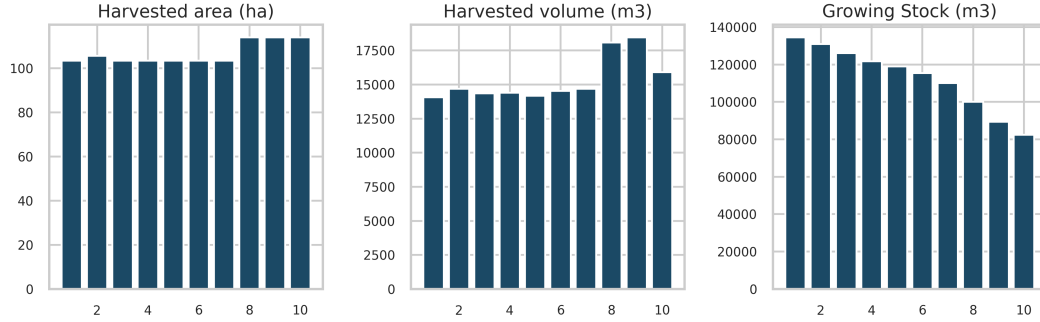


Figure 4: WS3 results: harvested area, harvested volume (utilization-adjusted), and growing stock over planning periods.

[14, 13], and climate impact assessment [17, 8, 19]. WS3 integrates with the SpaDES ecosystem for spatial simulation [4] and has been deployed as a backend in decision-support applications. The open MIT license, modular API, and PyPI distribution lower barriers to community contributions and reuse across jurisdictions.

Interoperability fosters reusability: tabular/text inputs, raster outputs, and the libCBM linkage allow WS3 to slot into existing analytical pipelines. The reproduction package, unit tests, and CI enhance trust and facilitate method transfer. The architecture is intentionally general, making it applicable to studies of sustainable yield, carbon accounting, biodiversity, and policy trade-offs [1].

5. Comparison to alternatives

WS3 differs primarily in its open Python API, explicit libCBM integration, and hybrid spatial/aspatial workflow designed for reproducible research and policy analysis. We emphasize that SIMFOR operates at stand level, while WS3, Woodstock, Patchworks, Heureka, and JLP target landscape-scale planning with varying degrees of spatial explicitness and availability [15, 18, 4, 7, 12, 16].

6. Conclusions and future work

WS3 provides an open, extensible framework for integrating harvest scheduling, simulation, and carbon accounting. By coupling solver-backed scheduling with libCBM via a documented interface, WS3 supports policy-relevant analyses with transparent, reproducible workflows.

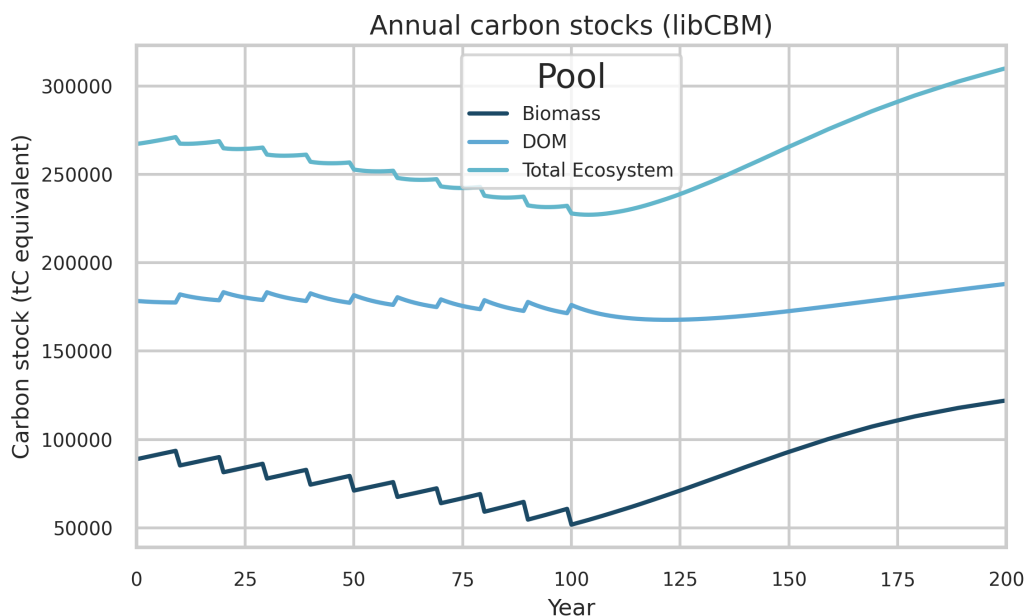


Figure 5: libCBM results: annual carbon stocks (biomass, dead organic matter, total ecosystem) over 200 years.

Future work will expand uncertainty handling (stochastic scheduling, sensitivity analysis), add cloud/HPC deployment patterns, and strengthen spatial allocation and visualization. We also plan to enhance carbon-economics linkages and support machine learning surrogates for speedups in large scenario ensembles. The open design invites community extensions and cross-ecosystem applications.

Code and data availability

WS3 source code is publicly available at <https://github.com/UBC-FRESH/ws3> and distributed via PyPI (`pip install ws3`). Versioned releases are archived on Zenodo; the EMS-paper release DOI will be provided in the final manuscript. Documentation and tutorials are available at <https://ws3.readthedocs.io>. A complete reproduction package for the case study, including environment files, scripts, and figure-generation notebooks, is provided as supplementary material and in the release archive.

Table 1: Comparison of WS3 with selected tools.

Tool	Scope	License	Optimization integration	Spatial flow	work-	Carbon gration	inte-	Geography fo-
WS3	Landscape	Open (MIT)	Built-in (PuLP; op- tional Gurobi)	Hybrid aspatial-to- raster		Built-in libCBM link- age		General
Remsoft Wood- stock	Landscape	Proprietary	Built-in	Extensions; connectors		External grations	inte-	General
Patchworks	Landscape	Proprietary	Built-in	Spatial ning emphasis	plan-	External grations	inte-	General
SpaDES	Simulation framework	Open	External pack- ages	Fully (agent-based, raster)	spatial	Via modules		General
SIMFOR	Stand-level	Open	Limited/none	N/A		Limited/none		General
Heureka	Landscape	Mixed/Restrict-	Built-in	Spatial bilities	capa-	External grations	inte-	EU-focused
JLP (MELA)	Landscape	Restricted/Legal	Built-in	Spatial sions	exten-	External grations	inte-	Finland- focused

Reproducibility. All figures and tables are generated by scripts in `papers/ems/repro`. The `make_repro.sh` script creates an isolated environment, installs pinned dependencies (`requirements.txt`), and runs `generate_diagrams.py`, a spatial schematic stub generator (`generate_spatial_stub.py`), and the case study script (`generate_case_study.py`). Outputs are written to `papers/ems/figs` and `papers/ems/tables`. The exact versions are preserved via the Zenodo-archived release.

FAIR compliance. Findable: versioned releases archived with DOI (Zenodo) and indexed on GitHub/PyPI. Accessible: open-source license (MIT) and public repository without authentication. Interoperable: open formats for tabular and raster data (CSV, GeoTIFF) and a documented API linking to libCBM. Reusable: pinned environments, CI-tested examples, detailed documentation, and a complete reproduction package accompanying this article.

CRedit authorship contribution statement

Gregory Paradis: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing—original draft, Writing—review & editing, Visualization, Supervision, Project administration, Funding acquisition.

Acknowledgements

Development of WS3 was supported by the Forest Resources Environmental Services Hub (FRESH) at the University of British Columbia. Funding was provided by the Canada Foundation for Innovation (CFI JELF), the British Columbia Knowledge Development Fund (BCKDF), Mitacs, Environment and Climate Change Canada (ECCC), the Natural Sciences and Engineering Research Council of Canada (NSERC), Natural Resources Canada (NRCan), and the Government of British Columbia. The author thanks Elaheh Ghasemi, Salar Ghotb, and Kathleen Coupland for their contributions to documentation, testing, and code quality improvements leading up to this release.

Declaration of competing interest

The authors declare no competing interests.

References

- [1] Boisvenue, C., Paradis, G., Eddy, I.M., McIntire, E.J., Chubaty, A.M., 2022. Managing forest carbon and landscape capacities. *Environmental Research Letters* 17, 114013.
- [2] Canadian Forest Service, 2024. libcbm-py: Python implementation of the carbon budget model. Python package. <https://pypi.org/project/libcbm-py/> (accessed 2025-09-28).
- [3] Cantegril, P., Paradis, G., LeBel, L., Raulier, F., 2019. Bioenergy production to improve value-creation potential of strategic forest management plans in mixed-wood forests of eastern canada. *Applied Energy* 247, 171–181.
- [4] Chubaty, A.M., McIntire, E.J.B., et al., 2024. Spades: Spatially explicit discrete event simulation. R package and framework. <https://spades.predictiveecology.org> (accessed 2025-09-28).

- [5] Gurobi Optimization, LLC, 2024. Gurobi Optimizer Reference Manual. <https://www.gurobi.com>.
- [6] Hall, J., et al., 2023. HiGHS: High performance software for linear optimization. Software. <https://highs.dev/> (accessed 2025-09-28).
- [7] Heureka Consortium, 2024. Heureka forestry decision support system. <https://www.heurekaslu.se/> (accessed 2025-09-28).
- [8] Ke, J., 2024. Climate impact assessment under different forest harvesting and fertilization scenarios. Master’s thesis. University of British Columbia.
- [9] Kurz, W., Dymond, C., Stinson, G., Rampley, G., et al., 2009. Cbm-cfs3: A model of carbon-dynamics in forestry and land use change. *Canadian Journal of Forest Research* 39, 495–509.
- [10] McIntire, E.J., Chubaty, A.M., Cumming, S.G., Andison, D., Barros, C., Boisvenue, C., Haché, S., Luo, Y., Micheletti, T., Stewart, F.E., 2022. Perfict: A re-imagined foundation for predictive ecology. *Ecology Letters* 25, 1345–1351.
- [11] Mitchell, S., et al., 2023. PuLP: A Linear Programming Toolkit for Python. Software. <https://coin-or.github.io/pulp/> (accessed 2025-09-28).
- [12] Natural Resources Institute Finland (Luke), 2024. Jlp (mela) forest planning system. [Http://mela2.metla.fi/mela/j/jlp-en.htm](http://mela2.metla.fi/mela/j/jlp-en.htm) (accessed 2025-09-28).
- [13] Paradis, G., Lebel, L., 2018a. Estimating the value-creation potential of wood supply using a hybrid simulation-optimization approach. Technical Report CIRRELT-2018-24. Centre interuniversitaire de recherche sur les réseaux d’entreprise, la logistique, et le transport (CIRRELT). URL: <https://www.cirrelt.ca/documentstravail/cirrelt-2018-24.pdf>.
- [14] Paradis, G., Lebel, L., 2018b. Retro-Fitting Value-Creation Potential Indicators to Long-Term Supply Models. CIRRELT-2018-23. URL: <https://www.cirrelt.ca/documentstravail/cirrelt-2018-23.pdf>.
- [15] Remsoft Inc., 2025. Woodstock. Software. <https://www.remsoft.com/products/woodstock/> (accessed 2025-09-28).

- [16] SIMFOR Project, 2020. Simfor: A stand-level simulation tool. Project site. (accessed 2025-09-28).
- [17] Smyth, C., Xu, Z., Lemprière, T., Kurz, W., 2020. Climate change mitigation in British Columbia’s forest sector: GHG reductions, costs, and environmental impacts. *Carbon Balance and Management* 15, 1–22.
- [18] Spatial Planning Systems Inc., 2025. Patchworks Forest Planning System. Software. <https://www.spatial.ca/patchworks/> (accessed 2025-09-28).
- [19] Yan, Y., 2025. A Mathematical Modeling Framework to Optimize the Climate Change Mitigation Potential of the Forest Sector in British Columbia, Canada. Master’s thesis. University of British Columbia.